

MACHINE LEARNING ENGINEERING

FASE 4 | AULA 02 -

BAG OF WORDS, EMBEDDINGS, WORD2VEC

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	20
REFERÊNCIAS.....	21

EMSE

O QUE VEM POR AÍ?

Os algoritmos não entendem texto. Dessa forma, precisamos transformá-los em representações numéricas, mas como? Como fazer com que o número contenha a informação que desejamos? Como manter a semântica entre as palavras? Como o algoritmo sabe que “rainha” é do gênero feminino, mais próximo a mulher e mais distante de homem? Esses são alguns dos desafios que a construção de variáveis em PLN enfrenta. Assim, vamos desbravar a construção de variáveis em PLN juntos(as)?

HANDS ON

Agora que sabemos pré-processar textos, veremos a construção das variáveis que alimentarão nosso modelo! Os algoritmos não entendem texto: eles precisam que nós transformemos o texto em representações numéricas para poder extrair padrões e realizar inferências.

Nessa aula, você vai aprender como transformar o texto em representações numéricas que servirão como variáveis para os algoritmos. Iremos abordar Bag of Words, Word2Vec e Embeddings.

SAIBA MAIS

Agora que já temos nossos dados limpos e pré-processados, precisamos transformá-los em uma representação numérica adequada para algoritmos de machine learning.

Para construir essas variáveis, podemos utilizar diferentes técnicas, das mais simples às mais complexas.

Técnicas de Representação de Texto

Bag of Words (BoW)

Representa cada documento como um vetor de frequências de palavras. Esse vetor será de números inteiros, visto que é uma contagem de palavras, e terá tamanho fixo, o tamanho do vocabulário passado. Essa é uma técnica bem simples de implementar, mas não considera a ordem das palavras nem a semântica entre elas.

Um dos problemas de não se considerar a ordem das palavras é a perda de informação. Por exemplo: “Maria gosta de João” e “João gosta de Maria” correspondem a vetores idênticos, com a mesma contagem de palavras, mas têm significados diferentes.

Vamos ver como funciona na prática? Para isso, usaremos a classe CountVectorizer da biblioteca sklearn.

```
from sklearn.feature_extraction.text import
CountVectorizer

corpus = [
    "Olá, mundo!",
    "Vamos desbravar o mundo de processamento de linguagem
natural?!",
    "Bem-vindos ao processamento de linguagem natural."
]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
print(X.toarray())
```

Saída:

```
['ao' 'bem' 'de' 'desbravar' 'linguagem' 'mundo' 'natural'
'olá'
'processamento' 'vamos' 'vindos']
[[0 0 0 0 0 1 0 1 0 0 0]
 [0 0 2 1 1 1 1 0 1 1 0]
 [1 1 1 0 1 0 1 0 1 0 1]]
```

Percebam que o [CountVectorizer](#) já possui alguns pré-processamentos embutidos que estão como default: transformação de letras para minúscula, encoding para utf-8, remoção de pontuação e tokenização a nível de palavra, entre outros. Além desses, há outros parâmetros bem úteis para pré-processamento que podem ser explorados, como `stop_words`, `max_features` e `vocabulary` que podem ser usados para limitar vocabulário, entre outros.

Um parâmetro bem interessante que podemos explorar é o `ngram_range`, criando uma bag of n-grams. Esse parâmetro ajuda na geração de contexto e no problema mencionado anteriormente da ordem das palavras. Usando-o, podemos criar conjuntos de tokens. Até agora vimos somente tokens de 1 gram, em que, por exemplo, uma palavra é um token, mas se quisermos criar tokens com combinações de até 2 palavras (bigrams) podemos usar o parâmetro `ngram_range = (1,2)`, resultando em tokens de 1 e 2 palavras. Vamos ver como fica o resultado?

```
from sklearn.feature_extraction.text import
CountVectorizer

corpus = [
    "Olá, mundo!",
    "Vamos desbravar o mundo de processamento de linguagem
natural?!",
    "Bem-vindos ao processamento de linguagem natural."
]
vectorizer = CountVectorizer(ngram_range=(1, 2))
X = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
print(X.toarray())
```

Saída:

```
['ao' 'ao processamento' 'bem' 'bem vindos' 'de' 'de
linguagem']
```

```

'de      processamento'   'desbravar'   'desbravar   mundo'
'linguagem'
'linguagem natural' 'mundo' 'mundo de' 'natural' 'olá'
'olá mundo'
'processamento'   'processamento   de'   'vamos'   'vamos
desbravar' 'vindos'
'vindos ao']
[[0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 1 0 0 0 0 0 0]
 [0 0 0 0 2 1 1 1 1 1 1 1 1 0 0 1 1 1 1 0 0]
 [1 1 1 1 1 1 0 0 0 1 1 0 0 1 0 0 1 1 0 0 1 1]]

```

Viu? Temos todos os mesmos tokens do unigrams e mais o bigram. Isso aumenta a quantidade de variáveis, o que pode aumentar a quantidade de recursos computacionais necessários. Mas mesmo resolvendo em parte o problema de ordenação das palavras, esse modelo ainda não considera contexto ou semântica das palavras e a distância entre os vetores nem sempre reflete a distância de significado.

TF-IDF

Ele pondera as palavras com base na frequência no documento e na inversa da frequência nos documentos. Dessa forma, esse algoritmo ajuda a reduzir o peso de palavras comuns e aumentar o peso de palavras distintivas.

Por exemplo: considere um algoritmo que classifica e-mails em spam ou não spam. Há palavras que estão presentes em quase todos os e-mails, como as stopwords que comentamos na aula de pré-processamento, além de outras como Atenciosamente, Abraços, Cordialmente e outras. Essas palavras não nos ajudam a identificar se o e-mail é spam ou não, certo?

Dessa forma, o TF-IDF entende que elas são usadas com frequência nos e-mails e atribui pesos baixos para elas. Já as palavras diferentes são aquelas que mais podem nos ajudar nessa classificação, recebendo pesos mais altos. Alguns exemplos comuns são: “economize muito dinheiro”, “cem por cento garantido”, “perder peso”.

$$TF(t, d) = \frac{f_{dt}}{l_d}$$

$$IDF(t) = (\log \frac{N}{n_t}) + 1$$

$$TFIDF = TF(t, d) * IDF(t)$$

Em que:

- d = documento;
- l_d = tamanho da sentença;
- f_{dt} = frequência que o termo t aparece na sentença;
- N = quantidade de documentos;
- n_t = número de documentos com o termo t .

Vamos calcular esses vetores juntos para o nosso exemplo?

Primeiro, vamos começar calculando o TF de cada palavra em relação ao documento:

palavra	documento 1	documento 2	documento 3
ao	0	0	1/7
bem	0	0	1/7
de	0	2/8	1/7
desbravar	0	1/8	0
linguagem	0	1/8	1/7
mundo	1/2	1/8	0
natural	0	1/8	1/7
olá	1/2	0	0
processamento	0	1/8	1/7
vamos	0	1/8	0
vindos	0	0	1/7

Tabela 1 - TF de cada palavra em relação ao documento
Fonte: Elaborado pelo autor (2024)

Agora vamos calcular o IDF:

palavra	IDF
ao	$\log(3/1) + 1 = 2,09$
bem	$\log(3/1) + 1 = 2,09$

de	$\log(3/2) + 1 = 1,405$
desbravar	$\log(3/1) + 1 = 2,09$
linguagem	$\log(3/2) + 1 = 1,405$
mundo	$\log(3/2) + 1 = 1,405$
natural	$\log(3/2) + 1 = 1,405$
olá	$\log(3/1) + 1 = 2,09$
processamento	$\log(3/2) + 1 = 1,405$
vamos	$\log(3/1) + 1 = 2,09$
vindos	$\log(3/1) + 1 = 2,09$

Tabela 2 - IDF
Fonte: Elaborado pelo autor (2024)

Dessa forma, temos:

palavra	IDF	TF Doc1	TF Doc2	TF Doc3
ao	2,09	0	0	0,14
bem	2,09	0	0	0,14
de	1,405	0	0,25	0,14
desbravar	2,09	0	0,125	0
linguagem	1,405	0	0,125	0,14
mundo	1,405	0,5	0,125	0
natural	1,405	0	0,125	0,14
olá	2,09	0,5	0	0
processamento	1,405	0	0,125	0,14
vamos	2,09	0	0,125	0
vindos	2,09	0	0	0,14

Tabela 3 – IDF e TF
Fonte: Elaborado pelo autor (2024)

Multiplicando TF por IDF para cada documento, chegamos nos vetores finais:

palavra	TF-IDF Doc1	TF-IDF Doc2	TF-IDF Doc3
ao	0	0	0,2926
bem	0	0	0,2926
de	0	0,35125	0,1967
desbravar	0	0,26125	0
linguagem	0	0,175625	0,1967
mundo	0,7025	0,175625	0
natural	0	0,175625	0,1967
olá	1,045	0	0
processamento	0	0,175625	0,1967
vamos	0	0,26125	0
vindos	0	0	0,2926

Tabela 4 – Vetores finais
Fonte: Elaborado pelo autor (2024)

E como calcular por código? Para isso podemos usar a classe `TfidfVectorizer` da biblioteca `sklearn`.

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(norm="l1", smooth_idf=False)
X = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
print(X.toarray())
```

Saída:

```
['ao' 'bem' 'de' 'desbravar' 'linguagem' 'mundo' 'natural'
'olá'
'processamento' 'vamos' 'vindos']
[[0.          0.          0.          0.          0.          0.
0.4010942
0.          0.5989058 0.          0.          0.          1]
```

[0.	0.	0.22255953	0.16616071	0.11127976
0.11127976				
0.11127976	0.	0.11127976	0.16616071	0.
[0.1760921	0.1760921	0.11793093	0.	0.11793093
0.11793093	0.	0.11793093	0.	0.1760921

Percebam que o TF-IDF não é uma contagem, então os vetores não são de números inteiros como o bag-of-words. Além disso, a classe `TfidfVectorizer` aplica algumas limpezas de dados, como considerar por padrão somente tokens com mais de 1 caractere alfanumérico. Você deve ter percebido que não temos um resultado para a palavra “o” no nosso exemplo.

Você deve ter percebido também que nossos resultados não ficaram iguais: isso se deve a uma normalização no final, que busca trazer os documentos para somarem 1. Outro ponto foi o parâmetro usado `smooth_idf`, caso verdadeiro, seria adicionado um à frequência dos documentos. Isso é usado para prevenir divisões por zero.

Assim como o BoW, também temos como usar n-grams no TF-IDF da mesma forma. Esse é um modelo que pode dar resultados melhores que um bag-of-words por penalizar palavras comuns que trazem pouca informação sobre o problema específico a ser resolvido, mas ainda sofre das mesmas fraquezas, falta de ordenação, significado semântico e alta dimensionalidade.

Word2Vec

Introduzido pelo Google, ele consiste em uma família de arquitetura de modelos e otimizações para elaboração de embeddings que cria vetores de palavras com base em contextos locais. Dessa forma, vetores distantes possuem significados diferentes e vetores próximos representam palavras com significados similares. Por exemplo: as palavras “mulher” e “rainha” estão perto no espaço vetorial, enquanto “mulher” e “muralha” estão mais distantes.

A premissa principal para conseguir criar representações vetoriais numéricas com contexto é definir o que é um contexto. Nessa família de arquitetura de modelos, o contexto consiste em algumas palavras antes e depois da palavra alvo. O contexto é definido através da decisão do tamanho da janela. Dessa forma, quanto mais uma palavra compartilha do mesmo contexto (palavras vizinhas) com outra palavra, mais similares elas são e mais perto no espaço vetorial elas ficarão.

Voltemos para o nosso exemplo, considerando a frase: "Bem-vindos ao processamento de linguagem natural.". Primeiramente, precisamos definir um tamanho de janela para criação dos pares.

Com uma janela de tamanho 2, teremos os seguintes pares:

```
"Bem-vindos ao processamento de linguagem natural."
pares: (ao, bem-vindos) (ao, processamento) (ao, de)
"Bem-vindos ao processamento de linguagem natural."
pares: (processamento, bem-vindos) (processamento,
ao) (processamento, de) (processamento, linguagem)
"Bem-vindos ao processamento de linguagem natural."
pares: (de, ao) (de, processamento) (de, linguagem) (de,
natural)
"Bem-vindos ao processamento de linguagem natural."
pares: (linguagem, processamento) (linguagem,
de) (linguagem, natural)
"Bem-vindos ao processamento de linguagem natural."
pares: (natural, de) (natural, linguagem)
```

É importante notar que quanto maior a janela, mais pares teremos. São esses pares que formarão nosso input e nosso label verdadeiro para criarmos um modelo de classificação.

Vamos ver algumas arquiteturas dessa família?

Continuous bag-of-words (CBow)

Previsão de uma palavra com base nas palavras adjacentes. Dessa forma, o objetivo de um modelo CBow é maximizar a probabilidade de prever a palavra alvo dado palavras de contexto.

O primeiro passo para o treinamento é tokenizar o vocabulário e vetorizá-lo através de uma transformação One hot encoder. Esses vetores sempre conterão somente um único 1 na palavra alvo e 0 no restante. Exemplo: na sentença: "Bem-vindos ao processamento de linguagem natural.", teríamos um vetor para o "Bem-vindos" que seria [1, 0, 0, 0, 0, 0], outro para o "ao" [0, 1, 0, 0, 0, 0] e assim por diante.

Após, esses vetores passam por uma camada de projeção (E) que é iniciada aleatoriamente, com tamanho $V \times D$, em que V é o total de palavras únicas e D é o tamanho de dimensão que você pode escolher. Essa será a matriz de pesos usada como camada de entrada. Multiplicando essa matriz com o vetor do one hot encoding, teremos os embeddings para cada palavra.

Agora fazemos a média desses vetores e multiplicamos por uma outra matriz de contexto E' de tamanho $D \times V$, resultando em uma matriz $1 \times V$, em que aplicando softmax chegamos ao resultado final. Comparamos o resultado com o vetor one hot encoded e retiramos o resultado da nossa função de perda (loss function), que em geral se utiliza o logloss.

Conforme comentamos, a tarefa do algoritmo é encontrar a palavra alvo das palavras de contexto. O gradiente é propagado para trás (backpropagated) e o modelo é treinado usando gradiente descendente. A figura 1 ilustra esse processo.

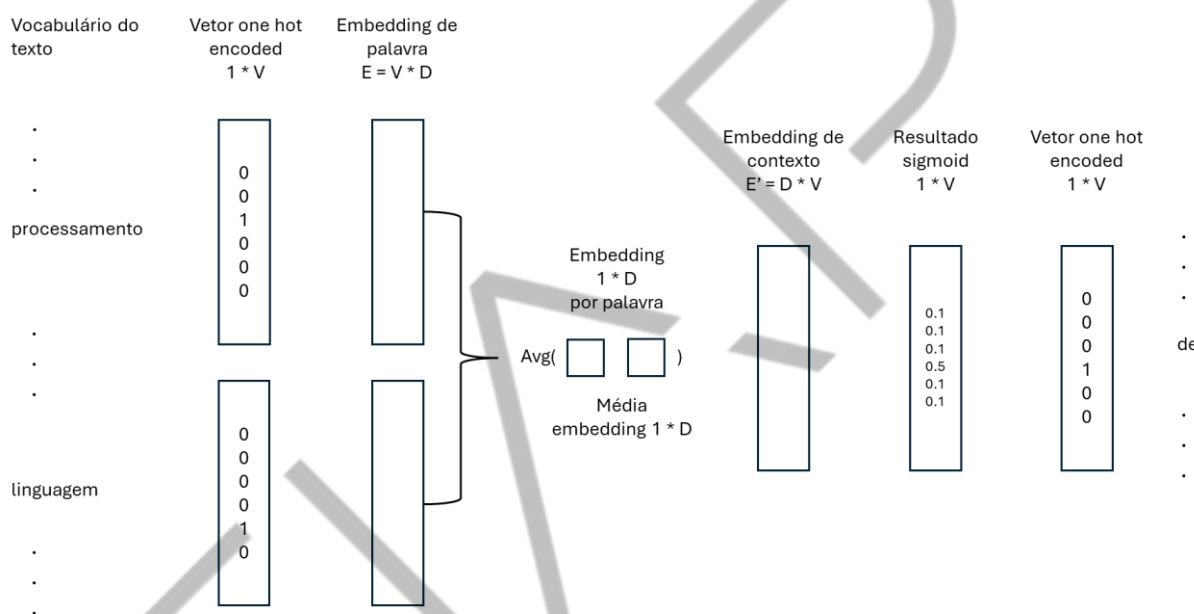


Figura 1 - Arquitetura CBoW
Fonte: Sen (2020)

Skip-gram

Previsão do contexto (palavras vizinhas) com base em uma palavra. Dessa forma, o contexto é representado por pares de skip-gram (palavra_alvo, palavra_contexto), em que a palavra_contexto aparece como vizinha da palavra palavra_alvo na sentença. Dessa forma, o objetivo de um modelo skip-gram é maximizar a probabilidade de prever palavras no contexto dada uma palavra alvo.

O processo é parecido com o inverso do CBoW. Começamos com o one-hot encoded da palavra do alvo, multiplicamos pelo embedding de palavra e usamos a matriz resultante na camada escondida da rede neural. O resultado da camada escondida é multiplicado pelo embedding de contexto. O resultado é então obtido

através da aplicação do softmax para o cálculo da função perda, normalmente uma logloss, e usando backpropagation com gradiente descendente para otimização.

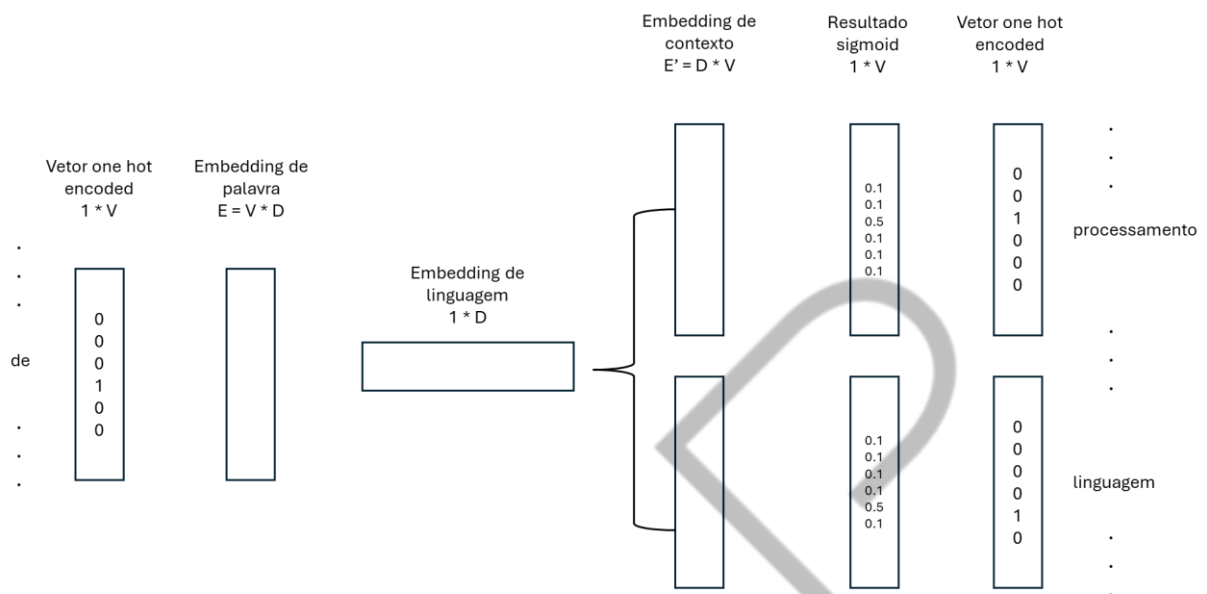


Figura 2 - Arquitetura Skip-gram
Fonte: Sen (2020)

Em ambos os casos, o que nos interessa é o Embedding de palavra E, que é armazenado e contém os pesos aprendidos durante a otimização e detém a relação entre as palavras com contextualização.

O modelo Skip-gram será melhor na previsão de palavras raras, enquanto o CBoW geralmente é melhor para palavras mais frequentes e é mais rápido de ser treinado.

A seguir mostramos como implementar Word2Vec usando a biblioteca [gensim](#).

Você talvez precise das [Ferramentas de compilação do Microsoft C++ - Visual Studio](#) para instalar o pacote gensim usando pip. Para isso, basta entrar no link e selecionar a opção “C++ build tools” e continuar a instalação.

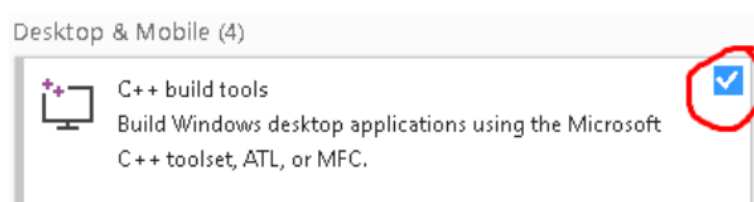


Figura 3 - C++ build tools
Fonte: Google Imagens (2024)

Após o treinamento do modelo, é interessante notar que os vetores são armazenados usando vetores chaves que podem ser chamados posteriormente, deixando o objeto menor e mais rápido. O modelo padrão da classe Word2Vec na biblioteca gensim é o CBoW.

```
from gensim.models import Word2Vec

sentencas = [["Olá", "mundo"], ["Bem-vindos", "ao",
"processamento", "de", "linguagem", "natural"]]
modelo = Word2Vec(sentencas, vector_size=10, window=2,
min_count=1, workers=4)
vetor_linguagem = modelo.wv['linguagem']
print(vetor_linguagem)
similar = modelo.wv.most_similar(['linguagem'], topn=1)
print(similar)
distancia_entre_palavras = modelo.wv.similarity('linguagem',
'natural')
print(distancia_entre_palavras)
```

Saída:

```
[ 0.07380505 -0.01533471 -0.04536613  0.06554051 -0.0486016
-0.01816018  0.0287658   0.00991874 -0.08285215 -0.09448818]
[('natural', 0.5436006188392639)]
0.54360056
```

Você pode ver que conseguimos extrair a similaridade semântica entre as palavras, tanto procurando/solicitando a palavra mais parecida com uma alvo como medindo a similaridade entre as palavras.

Anteriormente solicitamos a palavra mais similar à “linguagem” e o retorno foi “natural”. Percebam que para buscar a palavra mais similar foi utilizada a distância entre as palavras, a qual é calculada através da similaridade de cossenos dos vetores, que é o mesmo valor encontrado quando pedimos a similaridade entre “linguagem” e “natural”.

Embeddings

Embeddings são representações densas dos dados em vetores contínuos que capturam relações semânticas entre palavras, como sinônimos e contexto. Os

embeddings podem representar texto, vídeo, áudio e muito mais, mas nessa aula falaremos apenas sobre os embeddings de texto.

Embeddings de palavras

São representações vetoriais de palavras individuais que capturam o significado semântico das palavras com base em seu contexto de uso nos dados de treinamento.

Já vimos em Word2Vec exemplos de embeddings de palavras que buscam relações entre elas, criando um espaço vetorial no qual palavras parecidas ficam perto, enquanto palavras diferentes ficam afastadas. Nessa categoria ainda temos alguns modelos que valem ser mencionados:

- **FastText:** introduzido pelo Meta (Facebook), que estende o Word2Vec, mas usando tokenização por subpalavra para lidar com palavras raras e morfologias complexas; após, segue a arquitetura Skip-gram.
- **GloVe:** desenvolvido pela Stanford, combina métodos baseados em contagem e previsões.

No entanto, esses modelos não possuem informação de contexto, gerando uma única representação vetorial para cada palavra, independentemente do contexto em que ela aparece.

Dessa forma, esses embeddings são úteis para tarefas que envolvem palavras individuais, como similaridade de palavras, tradução automática e análise de texto.

Embeddings de contexto

Há outra categoria de embedding de palavras que consideram contexto, produzindo diferentes embeddings para uma palavra dependendo de seu contexto na frase. Isso é bastante importante, visto que a mesma palavra pode ter significados diferentes dependendo de seu contexto.

Por exemplo: “Ele sempre traz um boné na cabeça” e “Rodrigo é o cabeça da equipe.”. A palavra “cabeça” na primeira frase se refere a uma parte do corpo humano, enquanto na segunda se refere a liderança. Dessa forma, “cabeça” estaria mais perto da palavra “liderança” se a palavra “equipe” estiver por perto; com isso, “cabeça” possui mais de uma representação vetorial e qual embedding representará essa palavra dependerá das palavras de contexto.

Vamos ver alguns modelos que consideram contexto?

ELMo (Embeddings from Language Models)

Como funciona:

- **Embedding Inicial:** cada palavra é inicialmente representada por um vetor de embedding tradicional (como Word2Vec ou GloVe).
- **LSTMs Bidirecionais:** esses embeddings são passados por duas camadas de LSTM bidirecional, o que significa que há uma LSTM que processa a frase da esquerda para a direita e outra que processa da direita para a esquerda.
- **Combinação de Camadas:** as saídas dessas camadas são combinadas para formar o embedding final. A combinação é uma ponderação linear das representações das camadas, em que os pesos são aprendidos durante o treinamento da tarefa específica.

O ELMo representou um avanço significativo na modelagem de linguagens naturais ao fornecer embeddings contextuais profundos. A sua abordagem de usar LSTMs bidirecionais para capturar informações de contexto em múltiplas camadas permite uma representação mais rica e precisa de palavras em frases.

Embora existam modelos mais recentes, como BERT e GPT, que continuam a empurrar os limites do que é possível com embeddings contextuais, o ELMo permanece uma ferramenta poderosa e influente no campo do PLN.

BERT (Bidirectional Encoder Representations from Transformers)

O BERT também produz embeddings contextuais, mas usa a arquitetura Transformer em vez de LSTMs bidirecionais. Ele é treinado em uma tarefa de preenchimento de máscara (Masked Language Modeling) e uma tarefa de previsão de próxima sentença, permitindo capturar uma ampla gama de contextos e dependências. Veremos mais sobre sua arquitetura na próxima aula.

GPT (Generative Pretrained Transformer)

O GPT usa um Transformer unidirecional, processando a sequência de maneira unidirecional (da esquerda para a direita). Esse modelo é primariamente utilizado para

geração de texto, ao contrário de ELMo e BERT, que são mais voltados para tarefas de compreensão de linguagem. No módulo de AI Generativa falaremos mais sobre esse tipo de modelo.

Embeddings de sentença

São representações vetoriais de sentenças inteiras ou textos mais longos, capturando o significado global e o contexto do texto completo. Esses modelos já são sempre contextuais, levando em consideração toda a sentença para a geração do vetor.

Dessa forma, esses embeddings são úteis para tarefas que envolvem sentenças ou textos completos, como inferência textual, classificação de sentenças e medição de similaridade de sentenças.

Em geral, os embeddings de sentença tendem a ser mais complexos e de maior dimensão, pois precisam capturar informações de um texto completo.

Alguns exemplos incluem:

InferSent:

Método para gerar embeddings de sentenças desenvolvido pelo Meta (Facebook). É projetado para capturar o significado semântico de sentenças inteiras e tem se mostrado eficaz em várias tarefas de inferência de texto.

A arquitetura é bastante similar ao ELMo, mas em vez da representação a nível de palavra, representa uma sentença.

Entrada: o modelo recebe uma sentença e a representa como uma sequência de vetores de palavras (usualmente usando embeddings pré-treinados, como GloVe).

LSTMs Bidirecionais: a sequência de vetores de palavras é passada por uma camada de LSTM bidirecional. Isso significa que há duas LSTMs processando a sequência, uma da esquerda para a direita e outra da direita para a esquerda.

Concatenação de Estados: as saídas das LSTMs bidirecionais são concatenadas para formar a representação final da sentença.

Pooling: para obter um vetor fixo de representação da sentença, é aplicado um pooling max sobre todas as saídas das LSTMs.

Universal Sentence Encoder (USE)

Modelo treinado em uma grande variedade de tarefas de entendimento de linguagem para gerar embeddings universais de sentenças. Possui diferentes arquiteturas, uma baseada em transformers e outra baseada em Deep Averaging Networks (DANs).

Sentence-BERT (SBERT)

Modificação do BERT que usa redes de siamês para produzir embeddings de sentenças de forma eficiente, ideal para medição de similaridade de sentenças.

Como funciona?

- **Base do BERT:** usa um modelo pré-treinado do BERT como base.
- **Camada de Pooling:** após obter as representações de tokens do BERT, o SBERT adiciona uma camada de pooling para agregar a informação da sentença em uma única representação vetorial.
- **Redes Siamese/Triplet:** durante o treinamento, ele usa uma rede siamese para minimizar a distância entre sentenças similares e maximizar a distância entre sentenças diferentes. Isso é realizado com funções de perda específicas como triplet loss ou cosine similarity loss.

O SBERT é mais eficiente para tarefas de similaridade de sentenças porque gera embeddings fixos, enquanto o BERT requer processamento conjunto de pares de sentenças.

Em geral, modelos de embedding de palavras com contexto, como o BERT, permitem controle mais detalhado sobre o contexto, sendo opções geralmente melhores do que modelos de sentença para a maioria dos casos.

O QUE VOCÊ VIU NESTA AULA?

Nessa aula vimos diversas formas de transformar nossos dados em representações numéricas que podem ser usadas como variáveis em modelos de machine learning, como Bag-of-Words (BoW), TF-IDF, Word2Vec, BERT e muitos outros.

A escolha do método de transformação do texto em representações numéricas depende dos dados disponíveis, da complexidade do problema, da acuracidade desejada, dos recursos computacionais disponíveis e outros fatores.

Além disso, você viu como os embeddings podem ser poderosos por si só, já que permitem extrair diversas informações, como similaridade entre palavras, sentenças e qualquer tipo de texto.

REFERÊNCIAS

GÉRON, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. 2. ed. [s.l.]: O'Reilly Media, Inc., 2019.

MIKOLOV, T. et al. **Efficient Estimation of Word Representations in Vector Space**. 2013. Disponível em: <<https://arxiv.org/abs/1301.3781>>. Acesso em: 30 jul. 2024.

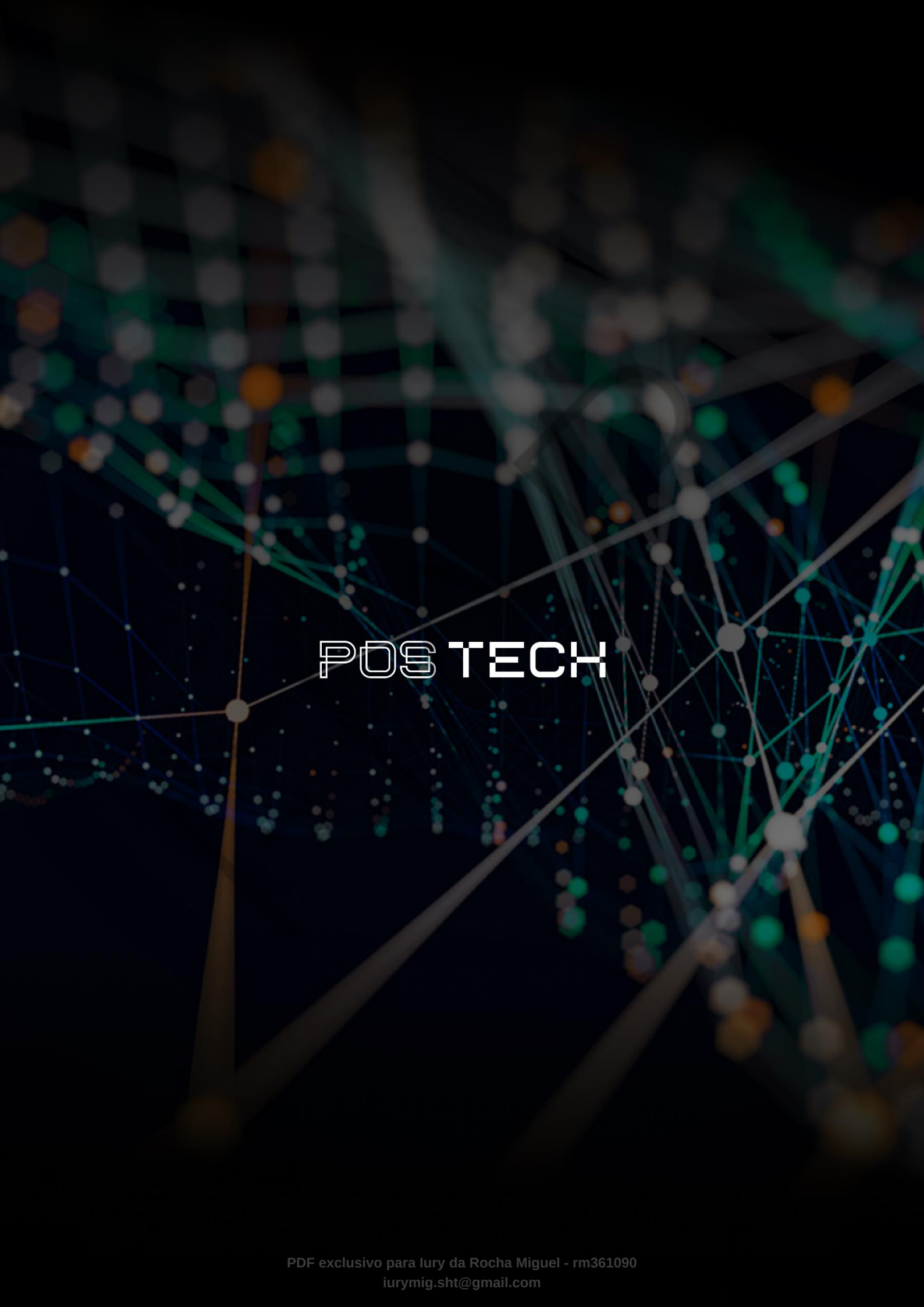
MIKOLOV, T. et al. **Distributed Representations of Words and Phrases and their Compositionality**. 2013. Disponível em: <<https://arxiv.org/abs/1310.4546>>. Acesso em: 30 jul. 2024.

SEN, A. **Text Classification — From Bag-of-Words to BERT — Part 2 (Word2Vec)**. 2020. Disponível em: <<https://medium.com/analytics-vidhya/text-classification-from-bag-of-words-to-bert-part-2-word2vec-35c8c3b34ee3>>. Acesso em: 30 jul. 2024.

PALAVRAS-CHAVE

Processamento de linguagem natural. Bag of words. Embeddings. Representação de texto. Machine learning.

EMAP

The background is a dark, abstract network visualization. It features a complex web of interconnected nodes and lines. The nodes are represented by small, semi-transparent circles in various colors, including teal, orange, and grey. The lines connecting them are thin, light-colored threads that create a sense of depth and connectivity. The overall effect is a futuristic, digital landscape.

POSTECH